

C++ OOP Concepts - Question Bank with Explanations

Question 1: Deep Copy

Question: Deep copy is achieved by:

- Default Constructor
- Copy Constructor
- Friend Constructor
- None

Answer: Copy Constructor

Explanation: A deep copy creates a new copy of dynamically allocated memory. This is typically achieved using a **copy constructor** that manually allocates new memory and copies the values from the source object. A shallow copy (default behavior) only copies pointers, not the data they point to, which can lead to issues like double deletion.

Question 2: Default Access Specifier

Question: What is default access specifier for data members or member functions declared within a class without any specifier, in C++?

- Private
- Protected
- Public
- Depends on the compiler

Answer: Private

Explanation: In C++, class members are **private by default**. This means if you don't specify an access specifier, all data members and member functions are private and cannot be accessed from outside the class. This differs from `struct`, where members are public by default.

Question 3: Variable Name Conflict

Question: Which is the most appropriate comment on following class definition?

```
cpp
```

```
class Student {  
    int a;  
    public: float a;  
}
```

- Error: same variable name can't be used twice
- Error: data types are different for same variable
- Error: Public must come first
- It is correct

Answer: Error: same variable name can't be used twice

Explanation: You cannot have two variables with the same name in the same class scope, even if they have different access specifiers or data types. This violates C++ naming rules and will result in a compilation error.

Question 4: Size of a Class

Question: Size of a class is:

- Sum of the size of all the variables declared inside the class
- Classes doesn't have any size
- Sum of the size of all the variables along with inherited variables in the class
- Size of the largest size of variable

Answer: Sum of the size of all the variables along with inherited variables in the class

Explanation: The size of a class object includes:

- All non-static data members
- Inherited data members from base classes
- Padding bytes (for alignment)
- Virtual table pointer (if virtual functions exist)

Static members and member functions do NOT contribute to object size.

Question 5: Friend Class

Question: Which of the following describes a friend class?

- Friend class can only access protected members of the class, of which it is a friend

- Friend class can access all the private members of the class, of which it is a friend
- Friend class don't have any implementation
- Friend class can't access any data member of another class but can use it's methods

Answer: Friend class can access all the private members of the class, of which it is a friend

Explanation: A **friend class** has special privileges to access private and protected members of the class that declared it as a friend. This breaks encapsulation but is useful in specific design scenarios where tight coupling is necessary.

Question 6: Wrong Class Syntax

Question: Which syntax for class definition is wrong?

- `class student{};`
- `class student{ public: student(int a){ } };`
- `class student student(int a){} };`
- `student class();`

Answer: Both `class student student(int a){} };` and `student class();`

Explanation:

- `class student student(int a){} };` - Invalid syntax, mixing class definition with function-like syntax
 - `student class();` - Invalid syntax, wrong order and format
 - The first two options are valid class definitions
-

Question 7: Class Features

Question: Which among the following is false for class features?

- Classes may/may not have both data members and member functions
- Classes may/may not have both data members and member functions (duplicate)
- Class can have only member functions with no data members
- Class is similar to union and structures

Answer: Class is similar to union and structures

Explanation: While classes, structures, and unions are all user-defined types, they are NOT similar:

- Classes have private members by default, structs/unions have public by default
 - Classes support full OOP features (inheritance, polymorphism)
 - Unions can only hold one member at a time
 - Classes can have only data, only functions, or both
-

Question 8: Abstract Class

Question: Instance of which type of class can't be created?

- Nested
- Parent
- Abstract
- None

Answer: Abstract

Explanation: An **abstract class** contains at least one pure virtual function (declared with = 0). You cannot create objects of abstract classes directly. They are meant to be base classes that define interfaces for derived classes to implement.

Question 9: Object Size Calculation

Question: What is size of the object of following class (64 bit system)?

```
cpp
class student {
    int rollno;
    char name[20];
    static int studentno;
};
```

- 20
- 22
- 24
- 26

Answer: 24

Explanation:

- `int rollno` = 4 bytes
- `char name[20]` = 20 bytes
- `static int studentno` = 0 bytes (static members don't contribute to object size)
- **Total = 24 bytes**

Note: On some systems, padding might affect this, but typically it's 24 bytes.

Question 10: Local Class

Question: If a local class is defined in a function, which of the following is true for an object of that class?

- Object is accessible outside the function
- Object can be declared inside any other function
- Object can be used to call other class members
- Object can be used/accessed/declared locally in that function

Answer: Object can be used/accessed/declared locally in that function

Explanation: A **local class** is defined inside a function and has local scope. Objects of this class can only be created and used within that function. They cannot be accessed outside the function where the class is defined.

Question 11: Object Assignment

Question: Which among the following is correct?

- `class student{ }s1,s2; s1.student()=s2.student();`
- `class student{ }s1; class topper{ }t1; s1=t1;`
- `class student{ }s1,s2; s1=s2;`
- `class student{ }s1; class topper{ }t1; s1.student()=s2.topper();`

Answer: `class student{ }s1,s2; s1=s2;`

Explanation: This is the correct syntax for:

1. Defining a class `student`
2. Declaring two objects `s1` and `s2`
3. Assigning `s2` to `s1` using the default assignment operator

The other options have syntax errors.

Question 12: Array of Objects Initialization

Question: Which among following is correct for initializing the class below?

```
cpp
class student{
    int marks;
    int cgpa;
public: student(int i, int j){
    marks=i;
    cgpa=j;
}
};
```

- `student s[2]={s(394,9), s(222,5) };`
- `student s[2]={s1(392,9), s2(222,5) };`
- `student s[2]={s[392,9], s2[222,5] };`
- None

Answer: None (Correct syntax would be: `student s[2]={student(394,9), student(222,5)};`)

Explanation: To initialize an array of objects with parameterized constructors, you must use the class name followed by the constructor parameters: `ClassName(params)`. None of the given options follow this correct syntax.

Question 13: Polymorphism

Question: If a function can perform more than 1 type of tasks, where the function name remains same, which feature of OOP is used here?

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Answer: Polymorphism

Explanation: Polymorphism means "many forms." In OOP, it allows a function to behave differently based on the context (function overloading) or the object type (function overriding). The same function name can

perform different tasks based on parameters or object type.

Question 14: Operator Overloading

Question: Which feature in OOP is used to allocate additional function to a predefined operator in any language?

- Operator Overloading
- Function Overloading
- Operator Overriding
- Function Overriding

Answer: Operator Overloading

Explanation: Operator overloading allows you to redefine the behavior of operators (like +, -, *, ==) for user-defined types (classes). For example, you can define what $\oplus$ means when adding two objects of a custom class.

Question 15: Encapsulation Definition

Question: Which among the following best describes encapsulation?

- It is a way of combining various data members into a single unit
- It is a way of combining various member functions into a single unit
- It is a way of combining various data members and member functions into a single unit which can operate on any data
- It is a way of combining various data members and member functions that operate on those data members into a single unit

Answer: It is a way of combining various data members and member functions that operate on those data members into a single unit

Explanation: Encapsulation is about bundling data and the methods that operate on that data together, while hiding the internal details from outside access. The key is that the functions specifically operate on the class's own data members.

Question 16: Accessing Object Members

Question: Figure out the correct statement:

cpp

```
class Car{
    public: int price;
};
int main(){
    Car c;
}
```

How to set price of the car object c to 9999?

- `c->price = 9999`
- `Car->price = 9999`
- `c.price = 9999`
- `Car.price = 9999`

Answer: `c.price = 9999`

Explanation: For a normal object (not a pointer), use the **dot operator** (.) to access members. The arrow operator (`->`) is used only with pointers to objects. `c` is an object variable, not a pointer, so we use `c.price`.

Question 17: Private Data Member Access

Question: What is the output:

```
cpp

class Student{
    int rollNo;
}
int main(){
    Student s;
    cout << s.rollNo;
}
```

- Garbage Value
- Compilation Error
- 0
- None

Answer: Compilation Error

Explanation: `rollNo` is a private member by default (no access specifier means private in a class). You cannot access private members from outside the class, so trying to access `s.rollNo` in `main()` will result in a **compilation error**.

Question 18: Accessing Private Data

Question: If data members are private, what can we do to access them from the class object?

- Create public member functions to access those data members
- Create private member functions to access those data members
- Create protected member functions to access those data members
- Private data members can never be accessed from outside the class

Answer: Create public member functions to access those data members

Explanation: The standard practice is to create **public getter and setter methods** (accessor and mutator functions) to provide controlled access to private data members. This maintains encapsulation while allowing necessary access.

Question 19: Static Member Variable

Question: What is the output of the code:

```
cpp
class Student(){
public:
    int rollNo;
    static int totalStudents;
    Student(){
        totalStudents++;
    }
};
int Student::totalStudents=20;
int main(){
    Student s1,s2,s3,s4;
    cout << Student::totalStudents;
}
```

- 20
- 24

- Compile Error
- None

Answer: 24

Explanation:

- `totalStudents` is initialized to 20
- Four objects are created (s1, s2, s3, s4)
- Each constructor increments `totalStudents` by 1
- Final value: $20 + 4 = 24$

Static members are shared among all objects of the class.

Question 20: Const Object

Question:

```
cpp
class Student{
public:
    int rollNo;
    int age;
};
int main(){
    Student s1;
    Student const s2 = s1;
    s1.rollNo=101;
    s1.age = 20;
    cout << s2.rollNo << " " << s2.age;
}
```

- 101 20
- Error
- Garbage Garbage
- None

Answer: Garbage Garbage

Explanation: When `s2` is created as a copy of `s1`, it gets the values of `s1` at that moment (which are uninitialized, hence garbage). Changes to `s1` after the copy don't affect `s2` because they are separate objects.

Since `s1` was never initialized before copying, `s2` contains **garbage values**.

Question 21: Encapsulation Example

Question: Find which of the following uses encapsulation?

- `void main(){ int a; void fun(int a=10; cout<<a); fun(); }`
- `class student{ int a; public: int b;};`
- `class student{int a; public: void disp(){ cout<<a;} };`
- None

Answer: `class student{int a; public: void disp(){ cout<<a;} };`

Explanation: This demonstrates **encapsulation** because:

- Private data member `a` is hidden
 - Public function `disp()` provides controlled access to the private data
 - The data and the function that operates on it are bundled together
-

Question 22: Encapsulation Violation

Question: Consider the following code and select the correct option.

```
cpp
class student {
    int marks;
    public: int* fun() {
        return &marks;
    }
};
main() {
    student s;
    int *ptr=c.fun();
    return 0;
}
```

- This code is good to go
- This code may result in undesirable conditions
- This code will generate error

- This code violates encapsulation

Answer: This code violates encapsulation

Explanation: Returning a pointer to a private member **violates encapsulation** because it allows direct external access to modify the private data. Anyone with this pointer can bypass the class's access control, defeating the purpose of making `marks` private.

Question 23: Encapsulation Maintenance

Question: Consider the code and select the wrong choice.

```
cpp
class hero {
    char name[10];
public: void disp() {
    cout<<name;
    }
};
```

- This maintains encapsulation
- This code doesn't maintain encapsulation
- This code is vulnerable
- This code gives error

Answer: This code doesn't maintain encapsulation (this is the WRONG choice)

Explanation: This code DOES maintain encapsulation correctly:

- `name` is private (hidden)
- `disp()` provides controlled read-only access
- No direct external access to private data

The statement "doesn't maintain encapsulation" is false, making it the wrong choice.

Question 24: Abstraction Usage

Question: If two classes combine some private data members and provides public member functions to access and manipulate those data members. Where is abstraction used?

- Using private access specifier for data members

- Using class concept with both data members and member functions
- Using public member functions to access and manipulate the data members
- Data is not sufficient to decide what is being used

Answer: Using public member functions to access and manipulate the data members

Explanation: Abstraction is about hiding implementation details and showing only essential features. When public functions provide an interface while hiding the internal workings (private data), abstraction is achieved. Users interact with the interface without knowing internal implementation.

Question 25: Destructor Sequence in Multiple Inheritance

Question: If in multiple inheritance, class C inherits class B, and Class B inherits class A. In which sequence are their destructors called if an object of class C was declared?

- $\sim A()$ then $\sim B()$ then $\sim C()$
- $\sim C()$ then $\sim A()$ then $\sim B()$
- $\sim C()$ then $\sim B()$ then $\sim A()$
- $\sim B()$ then $\sim C()$ then $\sim A()$

Answer: $\sim C()$ then $\sim B()$ then $\sim A()$

Explanation: Destructors are called in **reverse order** of construction.

- Construction order: $A \rightarrow B \rightarrow C$ (base to derived)
- Destruction order: $C \rightarrow B \rightarrow A$ (derived to base)

This ensures derived class cleanup happens before base class cleanup.

Question 26: Pointer to Object Syntax

Question: Which is correct syntax for declaring pointer to object?

- `*className objectName;`
- `className* objectName;`
- Both
- None

Answer: `className* objectName;`

Explanation: The correct syntax is `className* objectName;` or `className *objectName;`. This declares a pointer to an object of type `className`. The first option `*className objectName;` is incorrect syntax.

Question 27: Encapsulation vs Abstraction

Question: Encapsulation and abstraction differ as:

- Hiding and hiding respectively
- Binding and Hiding respectively
- Hiding and Binding respectively
- None

Answer: Binding and Hiding respectively

Explanation:

- **Encapsulation** = Binding/Wrapping data and methods together into a single unit
- **Abstraction** = Hiding implementation details and showing only essential features

They work together but serve different purposes in OOP.

Question 28: Memory Deallocation

Question: Which operator can be used to free the memory allocated for an object in C++?

- `unallocate()`
- `free()`
- `collect()`
- `delete`

Answer: `delete`

Explanation: In C++, the **delete operator** is used to free memory allocated with `new`.

- `delete ptr;` for single object
 - `delete[] ptr;` for arrays
 - `free()` is for C-style memory allocated with `malloc()`, not for objects created with `new`
-

Question 29: Constructor Selection

Question: Which constructor will be called from the object obj2 in the following C++ program?

```
cpp

class A {
    int i;
    A() { i=0; }
    A(int x) { i=x+1; }
    A(int y, int x) { i=x+y; }
};
A obj1(10);
A obj2(10,20);
A obj3;
```

- A(int y, int x)
- A(int y; int x)
- A(int y)
- None

Answer: A(int y, int x)

Explanation: obj2 is created with two integer arguments: A obj2(10,20); This matches the constructor A(int y, int x) which takes two parameters. The constructor will set $i = x + y = 20 + 10 = 30$.

Question 30: Object Creation Syntax

Question: Which among the following is correct for the class defined below?

```
cpp
```

```

class student {
    int marks;
public: student(){}
    student(int x) {
        marks=x;
    }
};

int main() {
    student s1(100);
    student s2();
    student s3=100;
    return 0;
}

```

- Program will give compile time error
- Object s3, syntax error
- Only object s1 and s2 will be created
- Program runs and all objects are created

Answer: Object s3, syntax error (Actually, only s1 and s3 are created correctly)

Explanation:

- `student s1(100);` ✓ Correct - calls parameterized constructor
- `student s2();` ✗ Most vexing parse - compiler treats this as a function declaration, not object creation
- `student s3=100;` ✓ Correct - implicit conversion/copy initialization

The correct answer should be that there's an issue with s2, not s3. The question may have errors, but the main learning point is the "most vexing parse" problem with `student s2();`.

Summary of Key OOP Concepts Covered

1. **Encapsulation:** Bundling data and methods together with access control
2. **Abstraction:** Hiding implementation details, showing only interfaces
3. **Polymorphism:** Multiple forms - function/operator overloading and overriding
4. **Inheritance:** Deriving classes from base classes
5. **Access Specifiers:** private (default for class), protected, public
6. **Constructors:** Default, parameterized, copy constructors

7. **Destructors:** Called in reverse order of construction
8. **Static Members:** Shared among all objects, don't contribute to object size
9. **Friend Classes:** Can access private members of other classes
10. **Abstract Classes:** Cannot be instantiated, contain pure virtual functions
11. **Const Objects:** Cannot modify member variables
12. **Pointers to Objects:** Use `->` operator instead of `.`
13. **Memory Management:** Use `new`/`delete` for dynamic allocation